

Back to the Future: Rethinking EDA Infrastructure for Agentic Systems in Chip Design Verification

Je Yang*

Department of Computer Science
Columbia University
New York, NY, USA
je.yang@cs.columbia.edu

Ivan Lobov

Google DeepMind
London, United Kingdom
lobov@google.com

Thomas Karpati

Google
Mountain View, CA, USA
tkarpati@google.com

Abstract

The unprecedented computational scale of modern artificial intelligence depends on complex, multi-billion-transistor Systems-on-Chip—yet the workflows that verify these chips remain stubbornly manual. Although Large Language Models (LLMs) have made rapid inroads into Electronic Design Automation (EDA), approximately 74.6% of existing studies target static Register-Transfer Level (RTL) code generation, leaving post-simulation verification and interactive waveform debugging largely untouched. We introduce **Back-to-the-Future (BTTF)**, an end-to-end agentic framework that closes this infrastructural gap. BTTF distills massive, unstructured simulation dumps into a normalized relational SQLite database and couples it with a collaborative multi-agent orchestration engine that translates natural-language verification queries into schema-aware SQL while correlating signal anomalies with versioned RTL repositories. Across a 150-query benchmark, BTTF attains 95.33% execution accuracy, charting a practical path toward autonomous EDA verification.

1 Introduction

Sustaining the computational demands of modern AI beyond the end of Moore’s law increasingly hinges on highly heterogeneous Systems-on-Chip (SoCs) [1]. As architectural complexity grows, design flows built on Hardware Description Languages (HDLs) become ever more labor-intensive, iterative, and error-prone [2]. Verification alone now consumes over 70% of the development cycle becoming the primary bottleneck of modern chip design [3], and functional bugs that slip into full-chip regression or post-silicon testing carry exponential financial and schedule penalties. Hence, automating register-transfer level (RTL) verification and analysis is an urgent, high-impact goal for the Electronic Design Automation (EDA) community.

The rapid advances of Large Language Models (LLMs) has naturally inspired a wave of AI-assisted EDA research [4, 5]. This wave, however, is strikingly lopsided. While approximately 74.6% of LLM-for-hardware studies concentrate on front-end RTL code generation and synthesis [6], post-simulation waveform debugging and dynamic verification remain largely underexplored. Since hardware development demands rigorous testing as much as design synthesis, this neglect leaves a substantial share of the automation opportunity untapped.

Even the few LLM-aided verification workflows that do exist operate almost exclusively on static source code or toy-scale simulation logs, whereas interactive post-simulation debugging is a fundamentally harder problem. When a dynamic simulation fails, engineers manually scrub through terabytes of temporal dumps in graphical waveform viewers, tracing illegal transitions back to their origin in the HDL hierarchy. Frontier LLM agents are locked out of this loop by what we term

*Work done while interning at Google.

Infrastructural Impedance. Raw, ASCII-based waveform dumps exceed model context windows by orders of magnitude, carry no structured semantic index, and expose no native, low-latency query interface that an autonomous agent could invoke.

Here, we propose **Back-to-the-Future (BTTF)**, an end-to-end agentic framework that enables natural-language querying for root-cause analysis and formal verification over complex RTL waveforms. Rather than forcing waveforms into an LLM’s context, BTTF revisits a classical EDA paradigm. It recasts unstructured temporal waveforms as a normalized relational SQLite database, then pairs this schema with a collaborative multi-agent engine that plans, queries, and audits on the engineer’s behalf. On 150 expert-annotated queries, BTTF reaches 95.33% overall accuracy, and its multi-agent design outcores a monolithic single-agent baseline by 17.3% on a five-criteria quality audit.

2 Related Work

LLM-Driven Hardware Verification. LLMs have demonstrated remarkable capacity across EDA research, assisting developers with bug localization, code generation, and automated verification [2, 4, 5, 7, 8, 9]. For complex verification tasks, state-of-the-art systems increasingly adopt multi-agent orchestration flows that split hardware synthesis into specialized roles for generation, testbench simulation, and formal counterexample refinement [5, 7, 10]. Yet these frameworks operate predominantly on static source code, compile-time assertion lists, or toy-scale simulation logs, implicitly assuming an LLM can localize functional errors from static HDL traces or terse compiler messages alone.

Key Limitation and Our Observation. The closest attempt to bridge this gap equips a coding agent with an Abstract Syntax Tree (AST)-based waveform tracing tool [11], but such traces remain tied to small, code-centric examples rather than production dumps. BTTF instead builds on a complementary observation: SQL generation is an emergent capability of LLMs, rooted in their deep semantic mapping from natural language to structured code [12]. Once waveforms are made *relational*, decades of database machinery (e.g., indexing, aggregation, low-latency filtering) become directly accessible to language agents.

3 BTTF Framework

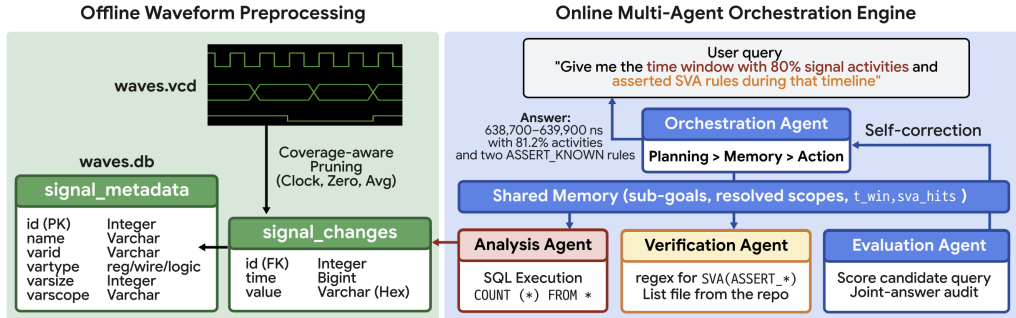


Figure 1: Overview of BTTF framework.

As Figure 1 illustrates, BTTF comprises two coupled subsystems on opposite sides of a single relational contract: (1) an offline **Waveform Preprocessing Pipeline** that compresses bulky simulation dumps into an optimized relational SQLite schema, and (2) an online **Agentic Orchestration Engine** that translates natural-language verification queries into schema-aware SQL and carries out the verification itself.

3.1 Waveform Preprocessing Pipeline

Standard EDA simulators emit proprietary binary waveforms (.fsdb) or ASCII Value Change Dump (.vcd) files; for SoC-scale blocks, a raw VCD text file routinely exceeds 10 GB—far beyond any prospect of direct context embedding. The BTTF pipeline therefore extracts the target hierarchies and normalizes them into a relational SQLite database (waves.db) built around two primary tables:

- `signal_metadata` captures the static signal declarations (RTL signal name and its variable type, etc.) with `id`.
- `signal_changes` records the temporal behavior as timestamped transitions as a `(time, value)` with a foreign key to refer metadata.

Separating *who a signal is* from *what it does* lets agents reason over structure and time independently, joining the two only when a query demands it. A naïve translation of every value change would simply move the scalability problem from the filesystem into the database. BTTF instead applies coverage-aware pruning during on-the-fly database generation:

- **Clock pruning.** High-frequency clock toggles dominate table size while carrying little debug information; since every row in `signal_changes` is timestamped, clock transitions can be skipped at insertion time without losing temporal anchoring.
- **On-the-fly inactive-signal pruning** (ZERO/AVG). To cope with duplicate VCD identifiers and inactive nets, the ingestion engine maintains a lightweight hash map (`vcd_id_to_db_id`) alongside a per-signal transition counter (`signal_change_count`). This bookkeeping enables two graduated policies: dropping statically silent zero-toggle wires (ZERO), or additionally dropping nets whose toggle rate falls below the design-wide average (AVG).
- **Batched insertion.** Row-by-row insertion incurs severe SQLite transaction overhead. Grouping transitions into batched writes removes this bottleneck at ingestion time.

3.2 LLM-Driven Agent Orchestration

On top of the generated `waves.db`, BTTF conducts verification through a collaborative multi-agent architecture. The main **Orchestration Agent** decomposes a natural-language query into sub-goals, routes them to specialized agents with own toolsets, and audits the merged answer.

- **Analysis Agent.** Armed with schema-aware multi-query execution tools, it composes SQLite queries over the two tables to handle scope discovery, event inspection, and toggle counting. For complex tasks such as power hotspot detection, it leverages an iterative function-calling loop to execute multi-turn queries and aggregate multi-cycle signal statistics.
- **Verification Agent.** Tying waveform anomalies back to version-controlled source by evaluating SVA rules (ASSERT, ASSERT_STATIC, ASSERT_NEVER, ASSERT_KNOWN, COVER) through the regular expression (REGEX) assignment.
- **Evaluation Agent.** Before any root-cause analysis is returned, this agent audits candidate queries against five criteria: (1) *correctness*—filtering, aggregation, and join fidelity to user intent; (2) *completeness*—inclusion of all required columns and time conditions; (3) *efficiency*—proper indexing and avoidance of redundant joins; (4) *schema adherence*—strict conformance to the two-table schema; and (5) *context understanding*—correct inference of implicit simulation timescales and hierarchy scopes.

Under the orchestrator, agents communicate solely through a dynamic session context maintained across turns; together with a self-correction loop on failed queries, this shared-memory discipline lets each agent stay narrowly specialized without losing global coherence. Furthermore, when an assertion rule is violated even after self-correction, the agent autonomously isolates the exact violation timestamp and pinpoints the problematic sub-signals responsible for the failure (see Appendix A.1 for detailed root-cause diagnostics).

4 Results

Experimental setup. All agents are served through the Gemini API with Gemini-2.5-Pro as the default model. Our benchmark spans multiple SoC design hierarchies of varying complexity (4.2 GB of VCD, 18,450 nets), from which we extract simulation windows ranging from narrow 50 ns high-resolution glitch-debug windows to 800 ns full-transaction simulation dumps. We curate 150 natural-language verification queries representative of daily engineering workflows, each paired with an expert-annotated golden answer.

Preprocessing Performance. We first quantify the cost of turning a waveform into an agent-ready database. As Table 1 shows, suppressing clock insertion alone shrinks the footprint by 66.20% and cuts query latency by $3.28\times$ while preserving 98.51% of functional toggle coverage. Additionally pruning untoggled signals pushes the reduction to 72.18%, while retaining full coverage over active nets. Aggressive below-average pruning compresses furthest but sacrifices 35.77% of coverage, marking the boundary of safe reduction. Relational schema optimization is thus not an

Pruning Strategy	Footprint	Reduc.	Latency	Coverage
Fully Unpruned	1,420 MB	0.00%	410.5 ms	100.00%
Clock Removed	480 MB	66.20%	125.2 ms	98.51%
Zero-Toggle Pruned	395 MB	72.18%	94.8 ms	100.00%
Below-Avg Pruned	185 MB	86.97%	41.6 ms	64.23%

Table 1: Trade-off of waveform preprocessing across pruning strategies.

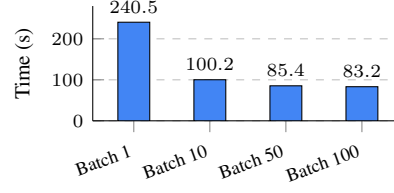


Figure 2: DB generation time speedup with batch insertion.

Table 2: Comprehensive taxonomy of verification tasks, example natural language user queries, and proof-of-concept execution accuracy.

Domain	Task / Query Type	Example Natural Language Query	Accuracy
Debugging	Scope/Net Discovery	“List all logics in the moduleA.”	100.00%
	RTL Attribution	“Find the RTL file related to logicA.”	87.50%
	Value Inspection	“What is the value of logicA at time timeA?”	95.00%
	Value Change Count	“How many times has logicA changed?”	100.00%
	Event Timing Check	“When is net logicA first set to valueA?”	100.00%
Analyzing	Toggle Coverage	“Calculate overall toggle coverage in jobA.db.”	91.67%
	Power Hotspot	“Give me the 100ns window with max signal activities.”	91.67%
Verification	Static SVA Check	“Verify ASSERT_STATIC(...).”	84.62%
	Dynamic SVA Check	“Run rule ASSERT_KNOWN(...).”	90.91%

implementation detail but a prerequisite for responsive agent interaction. Figure 2 confirms ingestion-side scalability. Batched insertion via SQL EXECUTEMANY accelerates database generation by up to $2.89\times$ over single-row insertion, removing the I/O bottleneck for multi-terabyte waveforms.

BTTF Framework Performance. Table 2 summarizes BTTF’s coverage across routine debugging, toggle-based performance analysis, and formal verification. Over the full 150-query taxonomy, BTTF achieves 95.33% overall accuracy—near-perfect on debugging primitives and consistently reliable on the harder analysis and verification tasks. A detailed breakdown of representative verification queries, agent tool invocations, and synthesized output responses across all task categories is provided in Appendix A.2 (Table 3).

We further compare a monolithic single-agent design against our collaborative architecture, with an autorater judging outputs on the Evaluation Agent’s five criteria. While a monolithic single agent achieves faster raw response times, it frequently suffers from context clutter, schema hallucination, and degraded causal tracing on multi-hop verification tasks, scoring only 3.35/5.00 on the Evaluation audit. In contrast, BTTF cleanly decouples schema-aware SQL generation from RTL source inspection and evaluation, achieving a 17.3% higher reasoning quality score and near-perfect execution accuracy. This specialized delegation incurs an average $1.96\times$ latency overhead, representing a practical and necessary trade-off to ensure robust, hallucination-free verification in production EDA workflows.

5 Discussion and Conclusion

Our BTTF argues, and empirically supports, a simple thesis. The binding constraint on agentic waveform debugging is data infrastructure, not model capability. By recasting terabyte-scale waveforms into a compact, vendor-agnostic relational schema, BTTF turns an intractable context problem into a routine text-to-SQL problem—one at which modern LLMs already excel—and its orchestration engine converts that access into grounded, audited root-cause analysis. The 95.33% benchmark accuracy and 17.3% quality gain from agent specialization suggest that autonomous EDA verification will be reached through better interfaces between agents and design data rather than ever-larger models alone. To scale across larger full-chip designs, streaming ingestion, standardized scope-path normalization, and a waveform-to-RTL symbol index are the natural next steps; evaluating larger frontier reasoning models to tackle multi-hop causal tracing over these stores remains an exciting direction for future work. More broadly, the recipe behind BTTF—normalize an opaque EDA

artifact into a queryable store, then let specialized agents operate over it—extends naturally beyond waveforms to regression logs, coverage databases, and power traces. We therefore view BTTF less as a point tool than as a template for rebuilding EDA infrastructure to meet agents halfway, and we hope it encourages the community to treat data interfaces as a first-class research object in AI-driven chip design.

References

- [1] Je Yang, Gracen Wallace, Joseph Zuckerman, Gabriele Tombesi, and Luca P. Carloni. A coherence-aware runtime reconfigurable system on chip for efficient language model inference. *IEEE Micro*, 46(3):33–42, 2026.
- [2] Yuchen Hu, Junhao Ye, Ke Xu, Jialin Sun, Shiyue Zhang, Xinyao Jiao, Dingrong Pan, Jie Zhou, Ning Wang, Weiwei Shan, et al. Uvllm: An automated universal RTL verification framework using LLMs. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025.
- [3] Cadence Design Systems. SoC verification: Tools, methodologies, and why it matters. https://www.cadence.com/en_US/home/explore/soc-verification.html, 2024.
- [4] Minwoo Kang, Mingjie Liu, Ghaith Bany Hamad, Syed M Suhaib, and Haoxing Ren. Fveval: Understanding language model capabilities in formal verification of digital hardware. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–6. IEEE, 2025.
- [5] Heng Ping, Arijit Bhattacharjee, Peiyu Zhang, Shixuan Li, Wei Yang, Anzhe Cheng, Xiaole Zhang, Jesse Thomason, Ali Jannesari, Nesreen Ahmed, et al. Verimoo: A mixture-of-agents framework for spec-to-hdl generation. *Proceedings of Machine Learning and Systems*, 8:1277–1290, 2026.
- [6] Meisam Abdollahi, Seyedeh Faegheh Yeganli, Mohammad Baharloo, and Amirali Baniasadi. Hardware design and verification with large language models: A scoping review, challenges, and open issues. *Electronics*, 14(1):120, 2024.
- [7] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. Mage: A multi-agent engine for automated RTL code generation. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025.
- [8] Antonio Mastropaolo, Luca Pascarella, Emanuela Guglielmi, Matteo Ciniselli, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. On the robustness of code generation techniques: An empirical study on github copilot. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2149–2160. IEEE, 2023.
- [9] Elisavet Lydia Alvanaki, Je Yang, Biruk Seyoum, and Luca P. Carloni. NoTB: Oracle-free triage of LLM-generated RTL via cross-model formal consensus, 2026.
- [10] Zaifeng Pan, AJKUMAR DAHYALAL PATEL, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVflow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *Advances in Neural Information Processing Systems*, 38:126246–126265, 2026.
- [11] Chia-Tung Ho, Haoxing Ren, and Bruce Khailany. Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 300–307, 2025.
- [12] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.

Table 3: Detailed taxonomy of verification queries, agent tool invocations, and synthesized output responses from benchmark evaluation.

Domain	Task / Query Type	Natural Language User Query	Agent Action with Tool Invocation Generated Output Response
Debugging	Scope/Net Discovery	“List all the nets in moduleA.”	Action: <code>find_signal_in_scope(db, "moduleA")</code> Output: <code>["bank_id", "bank_addr"]</code> (Found 2 struct field signals)
	Signal Scope Discovery	“What is the scope name of the signal port_id?”	Action: <code>find_scope_by_signal_name(db, "port_id")</code> Output: <code>moduleA_ep_{leaf, master, follower}_id</code>
	RTL Attribution	“Find the RTL file related to signal signalA.cnt.”	Action: <code>find_source_module("soc_top", ".../moduleA/rtl/", "cnt")</code> Output: <code>design/.../moduleA.sv:L142 (logic [7:0] cnt)</code>
	Value Inspection	“What is the value of signal ctrl_status at time 7155316?”	Action: <code>get_value_of_signal_at_time(db, "ctrl_status", 7155316)</code> Output: Value: 8184 (0x1FF8 in hex) at $t = 7,155.316$ ns
	Value Change Count	“What is the number of toggles of credit_stall?”	Action: <code>get_signal_toggle(db, "credit_stall", 6800000, 8000000)</code> Output: 4 toggles across simulation window [680 ns, 800 ns]
	Event Timing Check	“When is the first time the signal req_accept[7] is set to 42?”	Action: <code>get_first_time_of_signal_value(db, "req_accept[7]", "42", 181.5us, 184us)</code> Output: First transition detected at $t = 182,031.57$ ns (182031570 ps)
Analyzing	Toggle Coverage	“What is the toggle coverage of the scope pmu_events?”	Action: <code>get_scope_toggle_coverage(db, "pmu_events")</code> Output: 14.70% toggle coverage (21 active nets / 143 total nets in scope)
	Power Hotspot	“Give me a time range with the maximum signal activity.”	Action: Iterative rolling-window query across sliding 100 ns intervals Output: Peak activity window: [7612 ns, 9944 ns] with 27 total signal toggles
Verification	Static SVA Check	“Verify rule ASSERT_STATIC(req_mem_width, ...).”	Action: <code>verify_static_rule(rule_name, macro_def, rtl_params)</code> Output: PASSED ($64 == (48 + \lceil \log_2(65536) \rceil) \rightarrow 64 == 64, \text{True}$)
	Dynamic SVA Check	“Run the rule ASSERT_KNOWN(req_sel_is_known_A, req_sel, clk, rst).”	Action: <code>verify_known_rule(db, signal="req_sel", clk="clk", rst="rst_n")</code> Output: PASSED (Zero X/Z unknown states observed on req_sel at clock edges)

A Appendix: Detailed Query Execution Workflow and Output Analysis

A.1 Handling Assertion Failures and Root-Cause Diagnostics

In standard simulation logs, assertion checkers typically emit binary pass/fail flags without pinpointing the originating nets. Within BTTF, when an assertion rule (such as ASSERT_NEVER or ASSERT_KNOWN) fails during dynamic checking, the agent automatically extracts the exact violation timestamp $t_{\text{violation}}$ where the condition evaluates to true or where an invalid X/Z state is detected. It then initiates backward causal tracing by querying the prior values of all antecedent operands in the assertion expression at $t \leq t_{\text{violation}}$. Consequently, the final diagnostic response explicitly highlights the specific sub-signal responsible for the failure (e.g., “Assertion failed at $t = 6929$ ns due to net req_sel transitioning to unknown state x while clk was high”), bridging the gap between high-level assertion monitoring and low-level root-cause debugging.

A.2 End-to-End Query-to-Response Pipeline

Table 3 illustrates concrete examples of how the BTTF framework processes natural language user queries into structured tool executions and engineer-facing diagnostic answers. When an engineer submits a verification question (e.g., “List all the nets in moduleA”), the Orchestration Agent routes the query to the Analysis Agent. The agent leverages schema context to map the natural language scope into a structured function invocation (`find_signal_in_scope`) or a relational SQL join across `signal_metadata` and `changes`. By executing deterministic queries over the indexed `waves.db`, the system retrieves exact timestamps, signal values in hexadecimal or decimal format, and transition counts without context window truncation or string hallucination. Furthermore, when queries span both temporal waveform behavior and design intent (e.g., “Find the RTL file related to `signalA.cnt`”), the agent correlates the resolved hierarchy with the versioned repository to extract exact line numbers and declaration semantics.